

Métodos Numéricos - Laboratorio 1

Ricardo Largaespada

17 de marzo de 2026

1. Método de Bisección

El método de bisección, conocido también como de corte binario, de partición de intervalos o de Bolzano, es un tipo de búsqueda incremental en el que el intervalo se divide siempre a la mitad. Si la función cambia de signo sobre un intervalo, se evalúa el valor de la función en el punto medio. La posición de la raíz se determina situándola en el punto medio del subintervalo, dentro del cual ocurre un cambio de signo. El proceso se repite hasta obtener una mejor aproximación.

1.1. Criterios de paro y estimación de errores

Una sugerencia inicial sería finalizar el cálculo cuando el error verdadero se encuentre por debajo de algún nivel prefijado. Dicha estrategia es inconveniente, ya que la estimación del error en el ejemplo anterior se basó en el conocimiento del valor verdadero de la raíz de la función. Éste no es el caso de una situación real, ya que no habría motivo para utilizar el método si se conoce la raíz.

Por lo tanto, se requiere estimar el error de forma tal que no se necesite el conocimiento previo de la raíz. Se puede desarrollar una formulación alternativa en la aproximación del error relativo porcentual

$$\varepsilon_a = \left| \frac{x_u - x_l}{x_u + x_l} \right| 100 \% \quad (1)$$

Aunque el método de bisección por lo general es más lento que otros métodos, la claridad del análisis de error ciertamente es un aspecto positivo que puede volverlo atractivo para ciertas aplicaciones en ingeniería.

Otro beneficio del método de bisección es que el número de iteraciones requerido para obtener un error absoluto se calcula a priori. Debido a que en cada iteración se reduce el error a la mitad, la fórmula general que relaciona el error y el número de iteraciones, n , es

$$E_a^n = \frac{\Delta x^0}{2^n}.$$

Si $E_{a,d}$ es el error deseado, en esta ecuación se despeja

$$n = \log_2 \left(\frac{\Delta x^0}{E_{a,d}} \right) \quad (2)$$

1.2. Algoritmo de Bisección

El siguiente programa tiene en cuenta los siguientes aspectos:

- En caso que no se defina una tolerancia (error absoluto), el programa utilizará una por defecto.
- Si no se define la variable **filter**, que funciona para saber si en el intervalo dado exista una singularidad (esto es, que haya más de una raíz en el intervalo), el programa no lo considerará.

```
1 function root = bisect(func, x1, x2, filter, tol)
2 % Encuentra una raíz en un intervalo de f(x) por bisección.
3 % USAGE: root = bisect(func, x1, x2, filter, tol)
4 % INPUT:
5 %   func    - Manejador de función que retorna f(x).
6 %   x1, x2  - Límites del intervalo que contiene la raíz.
7 %   filter  - Filtro de singularidad: 0 = off (default), 1 = on.
8 %   tol     - Error de tolerancia (default es 1.0e4*eps).
9 % OUTPUT:
10 %   root    - Raíz de f(x), o NaN si se sospecha una singularidad.
11
12 % Si no se proporciona tol, se asigna el valor predeterminado
13 if nargin < 5
14     tol = 1.0e4 * eps;
15 end
16
17 % Si no se proporciona filter, se asigna 0 (desactivado)
18 if nargin < 4
19     filter = 0;
20 end
21
22 % Evaluamos la función en los extremos del intervalo
23 f1 = feval(func, x1);
24 if f1 == 0.0
25     root = x1; % Si x1 es raíz exacta, retornamos x1
26     return;
27 end
28
29 f2 = feval(func, x2);
30 if f2 == 0.0
31     root = x2; % Si x2 es raíz exacta, retornamos x2
32     return;
33 end
34
35 % Verificamos si la raíz está encerrada en el intervalo
36 if f1 * f2 > 0
37     error('La raíz no está encerrada en (x1, x2)');
38 end
39
```

```

40 % Calculamos el número máximo de iteraciones requeridas
41 n = ceil(log(abs(x2 - x1) / tol) / log(2.0));
42
43 % Método de bisección iterativo
44 for i = 1:n
45     x3 = 0.5 * (x1 + x2); % Punto medio del intervalo
46     f3 = feval(func, x3); % Evaluamos la función en el punto
        medio
47
48     % Si el filtro de singularidad está activado, detectamos
        valores sospechosos
49     if (filter == 1) && (abs(f3) > abs(f1)) && (abs(f3) > abs(f2)
        )
50         root = NaN;
51         return;
52     end
53
54     % Si encontramos la raíz exacta, la retornamos
55     if f3 == 0.0
56         root = x3;
57         return;
58     end
59
60     % Actualizamos los límites del intervalo según el signo de f(
        x3)
61     if f2 * f3 < 0.0
62         x1 = x3;
63         f1 = f3;
64     else
65         x2 = x3;
66         f2 = f3;
67     end
68 end
69
70 % Si no encontramos la raíz exacta, retornamos la mejor
    aproximación
71 root = (x1 + x2) / 2;
72 end

```

Ejemplo 1.1. Dada

$$f(x) = -2x^6 - 1.5x^4 + 10x + 2$$

Use el método de bisección para determinar el *máximo* de esta función. Haga elecciones iniciales de $x_l = 0$ y $x_u = 1$, y realice iteraciones hasta que el error relativo aproximado sea menor que 5 %.

Solución. Primero tengamos en cuenta que para encontrar el máximo de una función, debemos utilizar el criterio de la segunda derivada y evaluar si $f''(a)$ es mayor que 0, donde a es un número crítico. Así pues encontremos los números críticos de f , estos cumplen que:

$$f'(x) = -12x^5 - 6x^3 + 10 = 0.$$

Modificando el algoritmo de bisección anterior para que este se detenga hasta que e_a sea menor a 5 %.

```
1 function root = bisectej1(func, xl, xu, es, imax, xr)
2 % Encuentra una raíz en un intervalo de f(x) por el método de
   bisección.
3 %
4 % USAGE: root = bisectej1(func, xl, xu, es, imax, xr)
5 %
6 % INPUT:
7 %   func   - Manejador de función que retorna f(x).
8 %   xl, xu - Límites del intervalo que contiene la raíz.
9 %   es     - Nivel aceptable de error (%).
10 %   imax  - Número máximo de iteraciones permitidas.
11 %   xr    - Valor inicial arbitrario de la raíz (no afecta el cá
   lculo).
12 %
13 % OUTPUT:
14 %   root  - Aproximación de la raíz de f(x).
15
16 % Inicializamos error relativo aproximado (ea) y contador de
   iteraciones
17 ea = 100; % Se inicializa con 100% para garantizar la primera
   iteración
18 iter = 0; % Contador de iteraciones
19
20 % Bucle principal del método de bisección
21 while (1)
22     xrold = xr; % Guardamos el valor anterior de
   xr
23     xr = 0.5 * (xl + xu); % Calculamos el punto medio del
   intervalo
24     iter = iter + 1; % Incrementamos el contador de
   iteraciones
25
26     % Calculamos el error relativo aproximado (ea)
27     if xr ~= 0.0
28         ea = abs((xr - xrold) / xr) * 100; % Expresado en
   porcentaje
```

```

29     end
30
31     % Evaluamos el producto de f(xl) y f(xr) para determinar la
        ubicación de la raíz
32     test = feval(func, xl) * feval(func, xr);
33
34     if test < 0.0
35         xu = xr; % La raíz está en el intervalo [xl, xr]
36     elseif test > 0.0
37         xl = xr; % La raíz está en el intervalo [xr, xu]
38     else
39         ea = 0; % Se encontró la raíz exacta
40     end
41
42     % Condiciones de parada: si el error es menor al deseado o se
        alcanza imax
43     if ea < es || iter >= imax
44         break;
45     end
46 end
47
48 % Asignamos el valor final de xr como la raíz encontrada
49 root = xr;
50 end

```

El número crítico se obtiene al evaluar:

```

1 root = bisectej1(@(x) -12*x^5-6*x^3+10,0,1,5,3,10)

```

Obteniendo así: $root = 0.84375$, de donde el máximo valor de $f(x)$ es: 8.955640656873584.

2. Método de la Falsa Posición

Aún cuando la bisección es una técnica perfectamente válida para determinar raíces, su método de aproximación por “fuerza bruta” es relativamente ineficiente. La falsa posición es una alternativa basada en una visualización gráfica.

Un inconveniente del método de bisección es que al dividir el intervalo de x_l a x_u en mitades iguales, no se toman en consideración las magnitudes de $f(x_l)$ y $f(x_u)$. Por ejemplo, si $f(x_l)$ está mucho más cercana a cero que $f(x_u)$, es lógico que la raíz se encuentre más cerca de x_l que de x_u . Un método alternativo que aprovecha esta visualización gráfica consiste en unir $f(x_l)$ y $f(x_u)$ con una línea recta. La intersección de esta línea con el eje de las x representa una mejor aproximación de la raíz. El hecho de que se reemplace la curva por una línea recta da una “falsa posición” de la raíz; de aquí el nombre de *método de la falsa posición*, o en latín, *regula falsi*. También se le conoce como *método de interpolación lineal*.

Usando triángulos semejantes, la intersección de la línea recta con el eje de las x se estima mediante

$$\frac{f(x_l)}{x_r - x_l} = \frac{f(x_u)}{x_r - x_u} \quad (3)$$

en el cual se despeja x_r

$$x_r = x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)} \quad (4)$$

Ésta es la fórmula de la falsa posición. El valor de x_r calculado con la ecuación (??eq:3)), reemplazará, después, a cualquiera de los dos valores iniciales, x_l o x_u , y da un valor de la función con el mismo signo de $f(x_r)$. De esta manera, los valores x_l y x_u siempre encierran la verdadera raíz. El proceso se repite hasta que la aproximación a la raíz sea adecuada.

2.1. Algoritmo de la falsa posición

```
1: function FALSEPOS(xl, xu, es, imax, xr, 11:      if test<0 then
   iter, ea)                                12:          xu=xr
2:   iter = 0                               13:      else if test>0 then
3:   repeat                                  14:          xl=xr
4:       xrold=xr                            15:      else
5:       xr=xu-f(xu)*(xl-xu)/(f(xl)-f(xu)) 16:          ea=0
6:       iter=iter+1                         17:      end if
7:       if xr<>0 then                       18:      until ea<es OR iter ≥ imax
8:           ea=ABS((xr-xrold)/xr)*100      19:      FalsePos = xr
9:       end if                             20: end function
10:      test=f(xl)*f(xr)
```

3. Búsqueda por Incrementos y Determinación de Valores Iniciales

Además de verificar una respuesta individual, se debe determinar si se han localizado todas las raíces posibles. Como se mencionó anteriormente, por lo general una gráfica de la función ayudará a realizar dicha tarea. Otra opción es incorporar una búsqueda incremental al inicio del programa. Esto consiste en empezar en un extremo del intervalo de interés y realizar evaluaciones de la función con pequeños incrementos a lo largo del intervalo. Si la función cambia de signo, se supone que la raíz está dentro del incremento. Los valores de x , al principio y al final del incremento, pueden servir como valores iniciales para las técnicas descritas en esta clase.

Un problema potencial en los métodos de búsqueda por incremento es el de escoger la longitud del incremento. Si la longitud es muy pequeña, la búsqueda llega a consumir demasiado tiempo. Por otro lado, si la longitud es demasiado grande, existe la posibilidad de que raíces muy cercanas entre sí pasen inadvertidas. El problema se complica con la posible existencia de raíces múltiples. Un remedio parcial para estos casos consiste en calcular la primera derivada de la función $f'(x)$ al inicio y al final de cada intervalo. Cuando la derivada cambia de signo, puede existir un máximo o un mínimo en ese intervalo, lo que sugiere una búsqueda más minuciosa para detectar la posibilidad de una raíz.

Aunque estas modificaciones o el empleo de un incremento muy fino ayudan a resolver el problema, se debe aclarar que métodos tales como el de la búsqueda incremental no siempre resultan sencillos. Será prudente complementar dichas técnicas automáticas con cualquier otra información que dé idea de la localización de las raíces. Esta información se puede encontrar graficando la función y entendiendo el problema físico de donde proviene la ecuación.

A continuación un algoritmo desarrollado en Matlab que permite encontrar la menor raíz (si existe) en un intervalo cerrado de una función.

```
1 function [x1, x2] = rootsearch(func, a, b, dx)
2 % Búsqueda incremental para localizar una raíz de la función f(x)
3 %
4 % USO:
5 %     [x1, x2] = rootsearch(func, a, b, dx)
6 %
7 % ENTRADAS:
8 %     func — Manejador de la función que retorna f(x).
9 %     a, b — Límites del intervalo de búsqueda.
10 %     dx — Incremento de búsqueda (paso).
11 %
12 % SALIDAS:
13 %     x1, x2 — Intervalo que encierra la menor raíz dentro de (a, b)
14 %               ). Si no se detecta ninguna raíz, devuelve NaN.
15
16 % Inicialización del primer punto y evaluación de f(x)
17 x1 = a;
```

```

18     f1 = feval(func , x1);
19
20     % Inicialización del segundo punto y evaluación de f(x)
21     x2 = a + dx;
22     f2 = feval(func , x2);
23
24     % Iteración para encontrar un cambio de signo en f(x)
25     while f1 * f2 > 0.0
26         % Si hemos sobrepasado el límite derecho del intervalo
           sin encontrar una raíz
27         if x1 >= b
28             x1 = NaN;
29             x2 = NaN;
30             return;
31         end
32
33         % Avanzar al siguiente intervalo
34         x1 = x2;
35         f1 = f2;
36         x2 = x1 + dx;
37         f2 = feval(func , x2);
38     end
39
40 end

```


Problemas Propuestos

Problema 3.1. Determine las raíces reales de $f(x) = -0.5x^2 + 2.5x + 4.5$:

- a) Gráficamente
- b) Empleando la fórmula cuadrática
- c) Usando el método de bisección con tres iteraciones para determinar la raíz más grande. Emplee como valores iniciales $x_l = 5$ y $x_u = 10$. Calcule el error estimado e_a y el error verdadero e_t para cada iteración.

Problema 3.2. Determine la raíz real de $\ln(x^2) = 0.7$:

- a) Gráficamente
- b) Empleando tres iteraciones en el método de bisección con los valores iniciales $x_l = 0.5$ y $x_u = 2$.
- c) Empleando tres iteraciones del método de la falsa posición, con los mismos valores iniciales de b).

Problema 3.3. Encuentre la raíz positiva más pequeña de la función $x^2|\cos\sqrt{x}| = 5$ usando el método de la falsa posición. Para localizar el intervalo en donde se encuentra la raíz, grafique primero esta función para valores de x entre 0 y 5. Realice el cálculo hasta que ϵ_a sea menor que $\epsilon_s = 1\%$. Compruebe su respuesta final sustituyéndola en la función original.

Problema 3.4. La velocidad v de un paracaidista que cae está dada por

$$v = \frac{gm}{c} (1 - e^{-(c/m)t})$$

donde $g = 9.81 \text{ m/s}^2$. Para un paracaidista que con coeficiente de resistencia de $c = 15 \text{ kg/s}$, calcule la masa m de modo que la velocidad sea $v = 36 \text{ m/s}$ en $t = 10 \text{ s}$. Utilice el método de la falsa posición para determinar m a un nivel de $\epsilon_s = 0.1\%$.

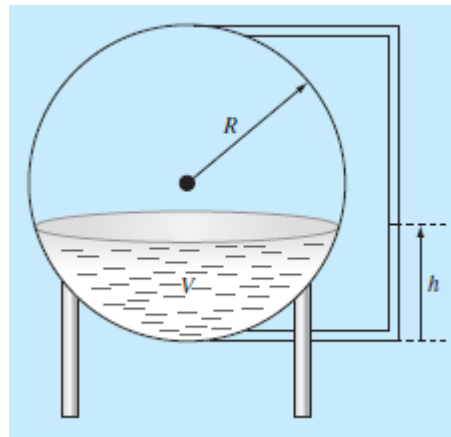
Problema 3.5. Use bisección para determinar el coeficiente de resistencia necesario para que un paracaidista de 82 kg tenga una velocidad de 36 m/s después de 4 s de caída libre. Comience con valores iniciales de $x_l = 3$ y $x_u = 5$. Itere hasta que el error relativo caiga por debajo de 2% .

Problema 3.6. Suponga que está diseñando un tanque esférico (ver figura) para almacenar agua para un poblado pequeño en un país en desarrollo. El volumen del líquido que se puede contener se calcula con

$$V = \pi h^2 \frac{[3R - h]}{3}$$

donde V = volumen [m^3], h = profundidad del agua en el tanque [m] y R = radio del tanque [m].

Si $R = 3 \text{ m}$, ¿a qué profundidad debe llenarse el tanque de modo que contenga 30 m^3 ? Haga tres iteraciones con el método de la falsa posición a fin de obtener la respuesta. Determine el error relativo aproximado después de cada iteración. Utilice valores iniciales de 0 y R .



Problema 3.7. Desarrolle un programa amigable para el usuario para el método de la falsa posición. La estructura del programa debe ser similar al pseudocódigo de la falsa posición que se bosquejó en la página 4. Pruebe el programa con la repetición del Problema 3.2.